

UNIVERSIDAD DEL VALLE DE GUATEMALA

CC3071 — Diseño de Lenguajes de Programación
Sección 31



Proyecto 2

Generador de Analizadores Sintácticos

Jose Auyón, 201579

Dilary Cruz, 231010

Nelson Escalante, 22046

GUATEMALA, mayo de 2026

Link de repositorio: https://github.com/auyjos/Generador_Sintactico

Link de vídeo: <https://youtu.be/wZlM286tqpQ>

El proyecto extiende el Generador Léxico desarrollado en el Proyecto 1, agregando una fase sintáctica completa. Mientras el generador léxico se encarga de transformar texto fuente en una secuencia de tokens, el generador sintáctico verifica si esa secuencia de tokens cumple con una gramática definida por el usuario.

Dado un archivo `.yalex` con la especificación de tokens y un archivo `.yapar` con la gramática del lenguaje, el sistema construye automáticamente tres analizadores sintácticos: SLR(1), LALR y LL(1). Además, presenta sus estructuras internas en un IDE interactivo, incluyendo el autómata LR(0), las tablas de parsing, los conjuntos FIRST/FOLLOW, el árbol de derivación y los pasos ejecutados durante el análisis.

El flujo general del sistema puede resumirse así:

```
Entrada de texto
-> Analizador lexico generado desde YALex
-> Secuencia de tokens
-> Analizador sintactico generado desde YAPar
-> ACCEPT / REJECT + arbol de derivacion
```

Fase 1 — Scanner YAPar (`yapar_scanner.py`)

El módulo `yapar_scanner.py` es la primera etapa del análisis sintáctico. Su función es leer el archivo `.yapar`, eliminar comentarios y extraer la información necesaria para construir una gramática formal.

El archivo `.yapar` contiene tres elementos principales:

- Declaración de tokens con `%token`.
- Declaración del símbolo inicial con `%start`.
- Producciones gramaticales delimitadas por `%%`.

La declaración de tokens indica cuáles son los terminales que el parser espera recibir desde el lexer. Por ejemplo:

```
%token ID NUMBER PLUS MINUS TIMES DIV LPAREN RPAREN
```

indica que la gramática utilizará tokens como identificadores, números, operadores y paréntesis. Estos nombres deben coincidir con los tokens producidos por la fase léxica.

La directiva `%start` define el símbolo inicial de la gramática. Este símbolo representa la estructura principal que el parser intentará reconocer. Por ejemplo:

```
%start E
```

indica que toda cadena válida debe poder derivarse desde el no terminal E.

Finalmente, las producciones indican cómo se pueden expandir los no terminales. Una producción como:

```
E : E PLUS T | T ;
```

significa que una expresión puede ser otra expresión seguida de una suma y un término, o simplemente un término.

Después de leer estas secciones, el scanner construye una especificación intermedia que posteriormente se convierte en un objeto **Grammar**. También valida que los símbolos usados en las producciones sean consistentes: los terminales deben haber sido declarados con `%token`, y los no terminales deben aparecer como cabezas de producciones.

```
def __init__(self, message: str, line: int | None = None):
    super().__init__(message)
    self.line = line

class VAParScanner:
    def __init__(self, filepath: str):
        self.filepath = filepath

    def scan(self) -> VAParSpec:
        with open(self.filepath, encoding='utf-8') as f:
            content = f.read()
            content = re.sub(r"/\*/\*/", "", content, flags=re.DOTALL)
            content = re.sub(r"/\*/", "", content)

        sections = content.split("\n")
        if len(sections) < 2:
            raise VAParScannerError("Missing \n separator in .yapar file")

        header, rules_section = sections[0], sections[1]
        tokens: list[str] = []
        ignore_tokens: list[str] = []
        start_symbol: str | None = None

        header_line_offset = content[:content.find("\n")].count('\n') if "\n" in content else 0
        for lineno, raw_line in enumerate(header.splitlines(), 1):
            line = raw_line.strip()
            if line.startswith("%token"):
                names = line[6:].split()
                if not names:
                    raise VAParScannerError("Empty token declaration", (line, lineno))
                tokens.extend(names)
            elif line.startswith("%ignore"):
                parts = line[6:].split()
                if parts:
                    start_symbol = parts[0]
            elif line.upper().startswith("%START"):
                ignore_tokens.extend(line.split()[1:])

        if not tokens:
            raise VAParScannerError("No token declarations found in header section")
        if not start_symbol:
            for block in rules_section.split(";"):
                block = block.strip()
                if not block:
                    continue
                colon_idx = block.find(':')
                if colon_idx != -1:
                    start_symbol = block[colon_idx:].strip()
                    break
```

Figura 1: Ejemplo de archivo .yapar

Fase 2 — Construcción del Autómata LR(0) (lr0.py)

A partir de la gramática construida por el scanner, el sistema genera el autómata LR(0). Este autómata representa los posibles estados del parser durante un análisis bottom-up. En lugar de partir del símbolo inicial y expandir hacia los tokens, como hace LL(1), los métodos LR consumen tokens y luego reducen fragmentos reconocidos hasta llegar al símbolo inicial.

El elemento central del autómata LR(0) es el **ítem LR(0)**. Un ítem es una producción con un punto que indica cuánto se ha reconocido de esa producción. Por ejemplo:

$$E \rightarrow T \bullet E'$$

significa que el parser ya reconoció un T y ahora espera reconocer E'.

Otro ejemplo:

$$E \rightarrow E \text{ PLUS } T \bullet$$

significa que la producción completa ya fue reconocida. En este caso, el parser puede aplicar una reducción.

La construcción del autómata depende de dos operaciones principales:

- **Closure:** si el punto está antes de un no terminal, se agregan todas las producciones de ese no terminal al estado actual.
- **GOTO:** dado un estado y un símbolo, mueve el punto sobre ese símbolo y calcula el estado siguiente.

La operación **closure** es necesaria porque, si el parser espera reconocer un no terminal, también debe considerar todas las formas posibles en que ese no terminal puede producirse. Por ejemplo, si un estado contiene:

$$E \rightarrow \bullet T$$

y T tiene las producciones:

$$T \rightarrow T \text{ TIMES } F$$

$$T \rightarrow F$$

entonces el closure agrega:

$$T \rightarrow \bullet T \text{ TIMES } F$$

$$T \rightarrow \bullet F$$

La operación **GOTO** permite avanzar en el análisis. Si un estado contiene:

$$E \rightarrow \bullet T$$

entonces al reconocer T , el punto avanza:

$$E \rightarrow T \bullet$$

La construcción completa inicia con la gramática aumentada:

$$S' \rightarrow \bullet S$$

donde S es el símbolo inicial original. A partir de este estado inicial, se aplican repetidamente **closure** y **GOTO** hasta generar todos los estados alcanzables del autómata.

Este autómata es la base que comparten SLR(1) y LALR, ya que ambos métodos dependen de los estados LR para construir sus tablas de parsing.

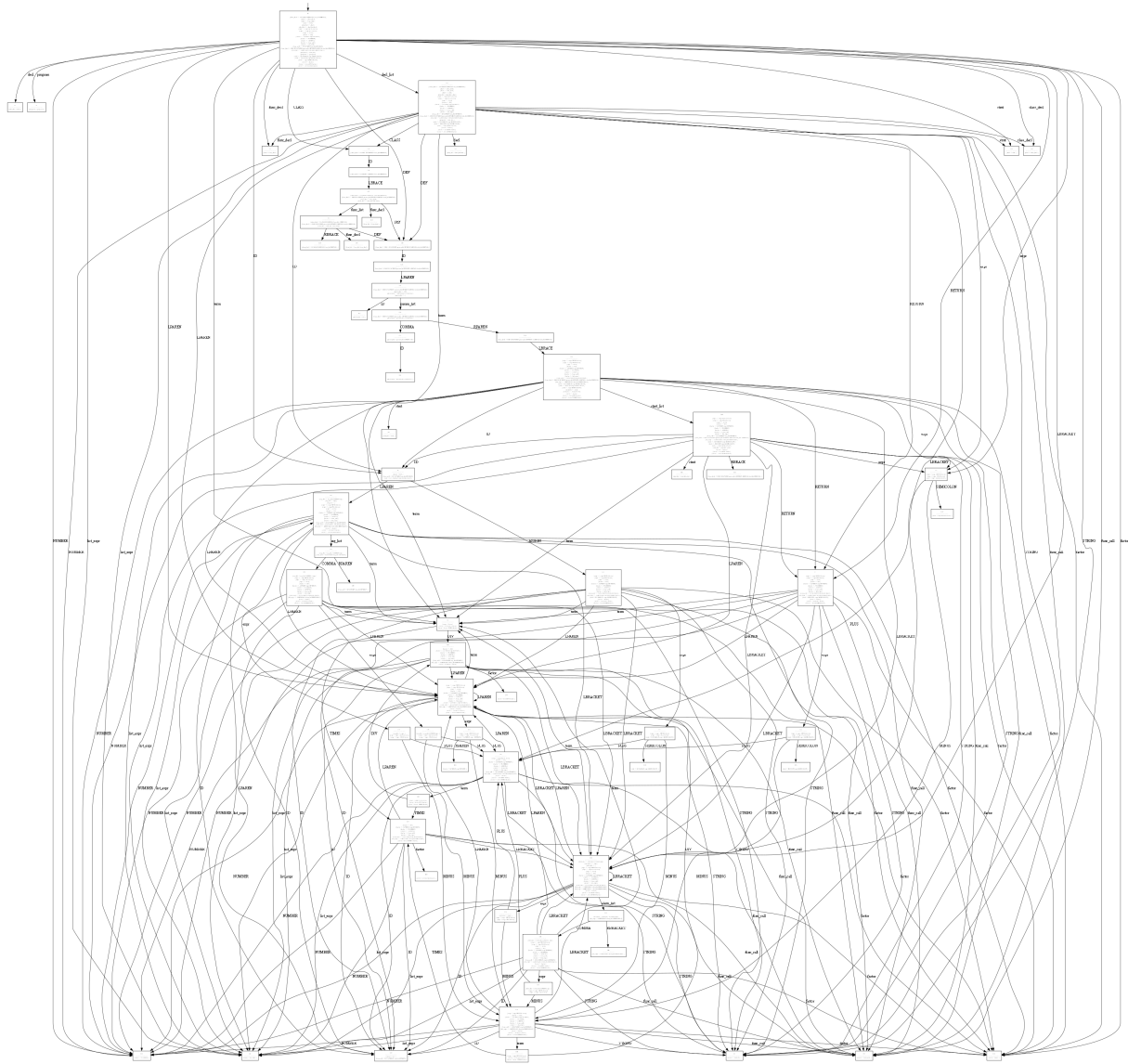


Figura 2: Autómata LR(0) generado

Fase 3 — Conjuntos FIRST y FOLLOW (first_follow.py)

Los conjuntos FIRST y FOLLOW son fundamentales para construir tablas de análisis sintáctico. Ambos conjuntos permiten razonar sobre qué terminales pueden aparecer en ciertas posiciones de una derivación.

- **FIRST(A)**: conjunto de terminales que pueden aparecer al inicio de alguna cadena derivada desde A.
- **FOLLOW(A)**: conjunto de terminales que pueden aparecer inmediatamente después del no terminal A en alguna forma sentencial.

Por ejemplo, para la producción:

```
F : LPAREN E RPAREN | ID | NUMBER ;
```

se obtiene:

$$FIRST(F) = \{LPAREN, ID, NUMBER\}$$

porque cualquier derivación de F comienza con uno de esos tres terminales.

Por otro lado, si existe la producción:

$$F \rightarrow LPAREN E RPAREN$$

entonces RPAREN pertenece a FOLLOW(E), porque puede aparecer inmediatamente después de E. Además, el símbolo inicial siempre incluye \$ en su conjunto FOLLOW, ya que \$ representa el final de la entrada.

Estos conjuntos son usados por distintos métodos:

- SLR(1) usa FOLLOW para decidir en qué terminales puede aplicar una reducción.
- LL(1) usa FIRST y FOLLOW para construir su tabla predictiva.
- LALR usa FIRST para calcular lookaheads más precisos en los ítems LR(1).

En la implementación, estos conjuntos se calculan iterativamente hasta alcanzar un punto fijo; es decir, el algoritmo repite las reglas de propagación hasta que ningún conjunto cambia.

```

1  You, 2 weeks ago | author (You)
2  from __future__ import annotations    You, 2 weeks ago • feat(parser): add YAPar syntactic analyzer gene...
3  from src.parser.grammar import Grammar, Symbol, EPSILON_SYM, EOF_SYM
4
5  def first_of_sequence(
6      seq: tuple[Symbol, ...],
7      first: dict[Symbol, set[Symbol]],
8  ) -> set[Symbol]:
9      result: set[Symbol] = set()
10     for sym in seq:
11         f = first.get(sym, set())
12         result |= f - {EPSILON_SYM}
13         if EPSILON_SYM not in f:
14             return result
15     result.add(EPSILON_SYM)
16     return result
17
18
19 def compute_first(grammar: Grammar) -> dict[Symbol, set[Symbol]]:
20     first: dict[Symbol, set[Symbol]] = {}
21     for t in grammar.terminals:
22         first[t] = {t}
23     first[EPSILON_SYM] = {EPSILON_SYM}
24     for nt in grammar.non_terminals:
25         first[nt] = set()
26
27     changed = True
28     while changed:
29         changed = False
30         for prod in grammar productions:
31             head = prod.head
32             old = len(first[head])
33             if not prod.body:
34                 first[head].add(EPSILON_SYM)
35             else:
36                 all_nullable = True
37                 for sym in prod.body:
38                     f = first.get(sym, set())
39                     first[head] |= f - {EPSILON_SYM}
40                     if EPSILON_SYM not in f:
41                         all_nullable = False
42                     break
43                 if all_nullable:
44                     first[head].add(EPSILON_SYM)
45             if len(first[head]) > old:
46                 changed = True
47     return first
48
49
50 def compute_follow(
51     grammar: Grammar,
52     first: dict[Symbol, set[Symbol]],
53 ) -> dict[Symbol, set[Symbol]]:
54     follow: dict[Symbol, set[Symbol]] = {nt: set() for nt in grammar.non_terminals}
55     follow[grammar.start].add(EOF_SYM)
56
57     changed = True
58     while changed:
59         changed = False
60         for prod in grammar productions:
61             for i, sym in enumerate(prod.body):
62                 if sym.is_terminal:
63                     continue
64                 if sym not in follow:
65                     follow[sym] = set()
66                 old = len(follow[sym])
67                 beta = prod.body[i + 1:]
68                 first_beta = first_of_sequence(beta, first)
69                 follow[sym] |= first_beta - {EPSILON_SYM}
70                 if EPSILON_SYM in first_beta:
71                     follow[sym] |= follow.get(prod.head, set())
72                 if len(follow[sym]) > old:
73                     changed = True
74     return follow

```

Figura 3: Tabla FIRST/FOLLOW visualizada en el IDE

Fase 4 — Parser SLR(1) (slr1.py)

El parser SLR(1) utiliza el autómata LR(0) y los conjuntos FOLLOW para construir una tabla ACTION/GOTO. Esta tabla determina qué debe hacer el parser en cada estado según el token actual.

La tabla se divide en dos partes:

- **ACTION:** define qué hacer cuando el parser está en un estado y observa un terminal.
- **GOTO:** define a qué estado pasar después de reducir a un no terminal.

Las acciones posibles son:

- **Shift:** consumir el token actual y pasar a un nuevo estado.
- **Reduce:** reemplazar una secuencia reconocida por el no terminal correspondiente.
- **Accept:** aceptar la cadena completa.
- **Error:** rechazar la cadena porque no existe una acción válida.

La construcción de la tabla sigue estas reglas:

- Si el punto está antes de un terminal a , se agrega una acción:

$$ACTION[s, a] = shift$$

- Si el punto está al final de una producción $A \rightarrow \alpha$, se agrega:

$$ACTION[s, t] = reduce(A \rightarrow \alpha)$$

para todo $t \in FOLLOW(A)$.

- Si se alcanza la producción aumentada completa:

$$S' \rightarrow S\bullet$$

con $\$$ como lookahead, se agrega la acción **accept**.

Durante el parseo, SLR(1) trabaja con una pila de estados. En cada paso consulta la tabla usando el estado en el tope de la pila y el token actual. Si la acción es **shift**, consume el token. Si es **reduce**, desapila los elementos del cuerpo de la producción y consulta **GOTO** para avanzar con la cabeza de la producción.

Por ejemplo, si se reconoce ID y existe la producción:

$$F \rightarrow ID$$

el parser puede reducir ID a F. Luego, si existe:

$$T \rightarrow F$$

puede reducir F a T, y finalmente reducir T a E.

Una limitación de SLR(1) es que usa FOLLOW de manera global. Esto puede generar conflictos en gramáticas donde un símbolo de FOLLOW es válido en algunos contextos, pero no en todos. Aun así, SLR(1) es útil porque es más simple que LALR y permite analizar muchas gramáticas con recursión izquierda.

Durante el parseo también se construye el árbol de derivación de forma paralela: cada **shift** crea un nodo hoja y cada **reduce** agrupa nodos bajo un nuevo padre.

	\$	ID	LPAREN	PLUS	RPAREN	TIMES	E	F	T
0		S2	S4				1	5	3
1	A0			S6					
2	R6			R6	R6	R6			
3	R2			R2	R2	S7			
4		S2	S4				8	5	3
5	R4			R4	R4	R4			
6		S2	S4					5	9
7		S2	S4					10	
8				S6	S11				
9	R1			R1	R1	S7			
10	R3			R3	R3	R3			
11	R5			R5	R5	R5			

Figura 4: Tabla SLR(1) en el IDE

Fase 5 — Parser LALR (lalr.py)

El parser LALR mejora la precisión de SLR(1) al utilizar lookaheads específicos por contexto. Mientras SLR(1) decide reducciones usando el conjunto FOLLOW completo de un no terminal, LALR utiliza información más precisa proveniente de ítems LR(1).

Un ítem LR(1) tiene la forma:

$$[A \rightarrow \alpha \bullet \beta, a]$$

donde a es un lookahead. Este lookahead indica en qué terminales tiene sentido aplicar una reducción o continuar el análisis en ese contexto particular.

La construcción LALR puede describirse en tres pasos:

1. Se construyen ítems LR(1), donde cada ítem conserva información de lookahead.
2. Se agrupan los estados que tienen el mismo núcleo LR(0). El núcleo LR(0) corresponde a las producciones y posición del punto, ignorando los lookaheads.
3. Se fusionan los lookaheads de los estados agrupados y se construye la tabla ACTION/GOTO.

Por ejemplo, los ítems:

$$[A \rightarrow \alpha \bullet, PLUS]$$

y

$$[A \rightarrow \alpha \bullet, RPAREN]$$

tienen el mismo núcleo:

$$A \rightarrow \alpha \bullet$$

Por lo tanto, pueden pertenecer al mismo estado LALR, conservando ambos lookaheads.

El resultado es una tabla con una cantidad de estados similar a LR(0), pero con decisiones de reducción más precisas que SLR(1). Esto reduce conflictos falsos, ya que LALR no reduce simplemente en todos los símbolos de FOLLOW, sino en los lookaheads que realmente corresponden al contexto del ítem.

En términos prácticos, LALR es un balance entre precisión y tamaño de tabla. Por eso es el enfoque usado tradicionalmente en herramientas como Yacc y Bison.

	\$	ID	LPAREN	PLUS	RPAREN	TIMES	E	F	T
0		S2	S4				1	5	3
1	A0			S6					
2	R6			R6	R6	R6			
3	R2			R2	R2	S7			
4		S2	S4				8	5	3
5	R4			R4	R4	R4			
6		S2	S4					5	9
7		S2	S4					10	
8				S6	S11				
9	R1			R1	R1	S7			
10	R3			R3	R3	R3			
11	R5			R5	R5	R5			

Figura 5: Tabla LALR en el IDE

Fase 6 — Parser LL(1) (ll1.py)

El parser LL(1) utiliza un enfoque diferente a SLR(1) y LALR. Mientras SLR y LALR son parsers bottom-up, LL(1) es un parser top-down. Esto significa que empieza desde el símbolo inicial y trata de expandirlo hasta coincidir con la entrada.

La notación LL(1) significa:

- Primera **L**: lee la entrada de izquierda a derecha.
- Segunda **L**: construye una derivación por la izquierda.
- **1**: utiliza un token de lookahead.

La tabla predictiva LL(1) se construye usando FIRST y FOLLOW:

- Para cada producción $A \rightarrow \alpha$, si $a \in FIRST(\alpha)$, se agrega:

$$M[A, a] = A \rightarrow \alpha$$

- Si $\varepsilon \in FIRST(\alpha)$, entonces para todo $b \in FOLLOW(A)$, se agrega:

$$M[A, b] = A \rightarrow \alpha$$

- Si dos producciones compiten por la misma celda, el sistema registra el conflicto y conserva la primera producción definida en la gramática.

Durante el análisis, el parser mantiene una pila de símbolos. Si el tope de la pila es un terminal y coincide con el token actual, se hace **match**. Si el tope es un no terminal, se consulta la tabla LL(1) para decidir qué producción usar.

Una limitación importante de LL(1) es que no puede manejar recursión izquierda. Por ejemplo, la producción:

$$E \rightarrow E \text{ PLUS } T$$

es problemática para LL(1), porque al intentar expandir E, el parser vuelve a generar E sin consumir ningún token. Esto puede producir un ciclo infinito. Por esta razón, el sistema incluye un límite de pasos para detectar este comportamiento y reportarlo de forma controlada.

Una gramática con recursión izquierda puede transformarse para LL(1) introduciendo no terminales auxiliares. Por ejemplo:

```
E : E PLUS T | E MINUS T | T ;
```

puede transformarse en:

```
E : T EPRIME ;

EPRIME : PLUS T EPRIME
        | MINUS T EPRIME
        |
        ;
```

Esta transformación elimina la recursión izquierda y hace que la gramática sea más adecuada para un parser predictivo.

FIRST / FOLLOW			SLR(1)	LALR	Productions	LL(1)
Not strict LL(1): 4 conflict(s) resolved by definition order (first production wins). Hover any conflicting cell for details.						
	ID	LPAREN				
E	E -> E PLUS T	E -> E PLUS T				
F	F -> ID	F -> LPAREN E RPAREN				
T	T -> T TIMES F	T -> T TIMES F				

Figura 6: Tabla LL(1) en el IDE

Fase 7 — Árbol de Derivación (parse_tree.py)

El árbol de derivación representa cómo una cadena aceptada puede derivarse desde el símbolo inicial de la gramática. Es una evidencia visual de que el parser no solo aceptó la cadena, sino que también encontró una estructura gramatical válida.

Cada parser construye el árbol de acuerdo con su estrategia:

- **SLR(1) / LALR (bottom-up):** el árbol se construye desde las hojas hacia la raíz. Cada **shift** crea un nodo hoja con el token leído. Cada **reduce** toma los nodos correspondientes al cuerpo de una producción y los agrupa bajo un nodo padre con el nombre del no terminal.
- **LL(1) (top-down):** el árbol se construye desde la raíz hacia las hojas. Cada expansión de un no terminal crea sus hijos según la producción seleccionada por la tabla predictiva.

Por ejemplo, para una reducción:

$$F \rightarrow ID$$

el parser bottom-up crea un nodo **F** cuyo hijo es el token **ID**:

F

ID

Luego, si se aplica:

$$T \rightarrow F$$

se crea un nodo **T** sobre **F**, y así sucesivamente hasta llegar al símbolo inicial.

La visualización se genera con Graphviz. En el árbol, los no terminales se representan como nodos internos y los terminales como hojas. Esto permite inspeccionar claramente qué producciones fueron aplicadas durante el análisis.

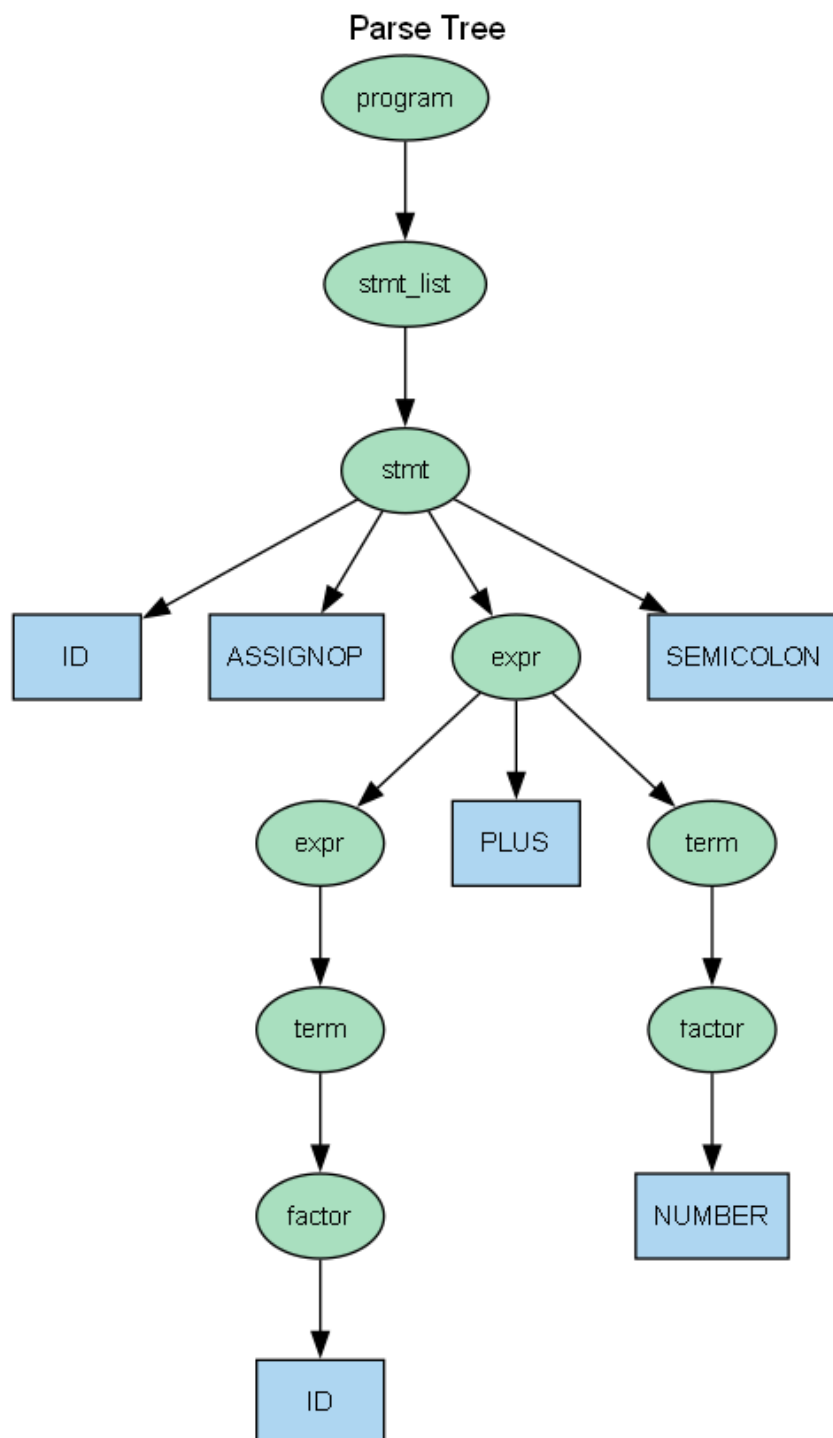


Figura 7: Árbol de derivación para una cadena aceptada

Fase 8 — Manejo de Errores (`string_analyzer.py`)

El módulo `string_analyzer.py` coordina el análisis de cadenas y mejora los mensajes de error generados por los parsers. Su propósito no es únicamente decir si una cadena fue rechazada, sino también indicar en qué parte ocurrió el problema.

El sistema reporta errores con ubicación precisa mediante dos mecanismos:

1. **Token spans:** durante la tokenización, se guarda la posición de inicio y fin de cada token dentro del texto original.
2. **Enriquecimiento del error:** cuando un parser falla, se utiliza la posición del token problemático para agregar columna y contexto al mensaje.

Por ejemplo, para una entrada inválida como:

```
x + * 5
```

el lexer puede producir:

```
ID PLUS TIMES NUMBER
```

Sin embargo, sintácticamente la cadena es inválida porque después de PLUS se esperaba un factor, como ID, NUMBER o LPAREN, pero apareció TIMES.

Un mensaje de error enriquecido puede tener la forma:

```
Unexpected '*' for 'F'. Expected: ['ID', 'NUMBER', 'LPAREN']
-> column 5, near '*'
```

Esto hace que el sistema sea más útil para depuración, porque el usuario no solo sabe que la cadena fue rechazada, sino también dónde ocurrió el error y qué se esperaba en ese punto.

```

=====
ANALYSIS RESULTS
=====

[!] LL(1): 4 conflict(s) resolved by definition order (first wins):
M[E,ID]: kept E -> E PLUS T (defined first), skipped E -> T
M[E,LPAREN]: kept E -> E PLUS T (defined first), skipped E -> T
M[T,ID]: kept T -> T TIMES F (defined first), skipped T -> F
M[T,LPAREN]: kept T -> T TIMES F (defined first), skipped T -> F

[01] A + B
Tokens : ['ID', 'PLUS', 'ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[02] (A + B) * C
Tokens : ['LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN', 'TIMES', 'ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[03] x * (y + z)
Tokens : ['ID', 'TIMES', 'LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[04] (A * B) + (C * D)
Tokens : ['LPAREN', 'ID', 'TIMES', 'ID', 'RPAREN', 'PLUS', 'LPAREN', 'ID', 'TIMES', 'ID', 'RPAREN']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[05] a + b + c
Tokens : ['ID', 'PLUS', 'ID', 'PLUS', 'ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[06] (x + y) * (z + w)
Tokens : ['LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN', 'TIMES', 'LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[07] A * B * C
Tokens : ['ID', 'TIMES', 'ID', 'TIMES', 'ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[08] (a + b) * (c + d) * e
Tokens : ['LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN', 'TIMES', 'LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN', 'TIMES', 'ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[09] x
Tokens : ['ID']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[10] (A)
Tokens : ['LPAREN', 'ID', 'RPAREN']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[11] a + (b * (c + d))
Tokens : ['ID', 'PLUS', 'LPAREN', 'ID', 'TIMES', 'LPAREN', 'ID', 'PLUS', 'ID', 'RPAREN', 'RPAREN']
SLR(1) : ACCEPT | LALR: ACCEPT | LL(1): REJECT

[12] (X + Y) * Z - W
Tokens : []
SLR(1) : REJECT | LALR: REJECT | LL(1): N/A
[!] Error : Lex error: Lexer error at column 13: unexpected character '-'

```

Figura 8: Error con columna exacta en el panel de resultados

Fase 9 — Interfaz Gráfica IDE (app.py)

La interfaz gráfica fue desarrollada con PyQt6 y funciona como un IDE para cargar, analizar y visualizar archivos `.yalex`, `.yapar` y cadenas de prueba. Su objetivo es facilitar la interacción con el sistema y mostrar de forma visual las estructuras generadas por los algoritmos.

La GUI presenta los resultados en pestañas:

- **Editor:** permite cargar archivos y escribir entradas manuales.
- **LR(0):** muestra el autómata LR(0) generado a partir de la gramática.
- **Tables:** presenta FIRST/FOLLOW, la tabla SLR(1), la tabla LALR, la tabla LL(1) y la lista de producciones.
- **Parse Tree:** muestra el árbol de derivación de una cadena aceptada.
- **Steps:** permite revisar el análisis paso a paso, mostrando acciones como `shift`, `reduce`, `expand` o `match`.
- **Results:** muestra los resultados ACCEPT/REJECT para cada cadena, junto con mensajes de error cuando corresponde.

Esta interfaz tiene valor didáctico porque permite observar directamente cómo cambian las pilas del parser, qué acción se aplica en cada paso y cómo se llega a la aceptación o rechazo de la cadena. También permite comparar los resultados de SLR(1), LALR y LL(1) sobre una misma entrada.

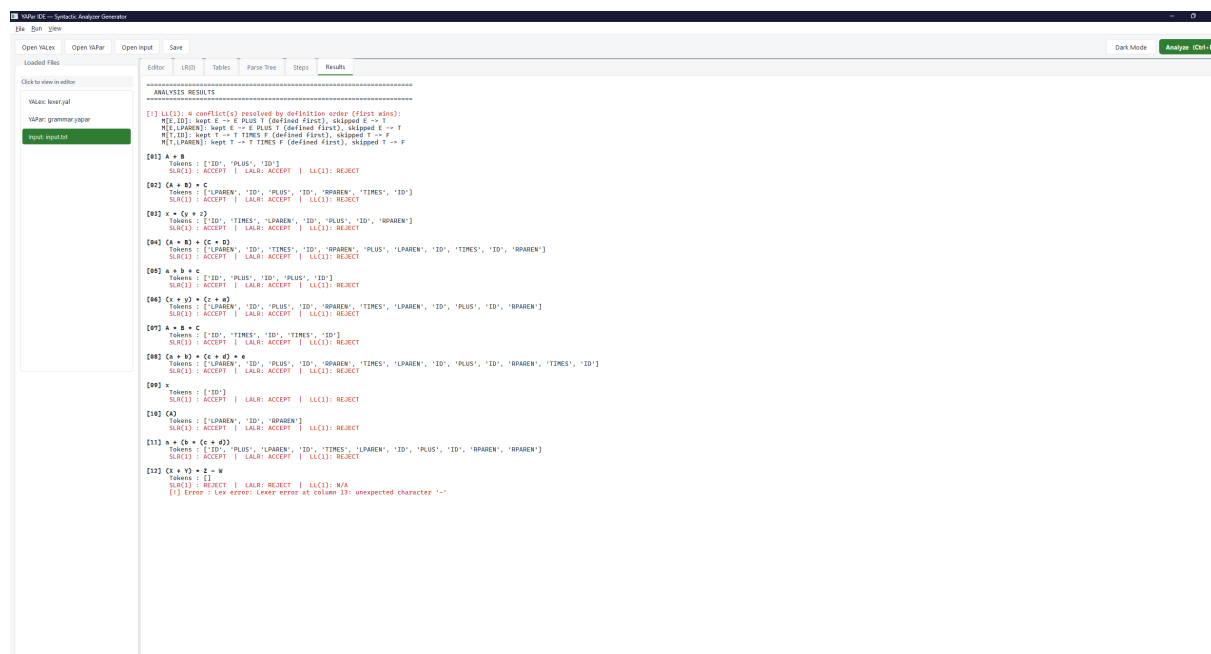


Figura 9: IDE completo del Generador de Analizadores Sintácticos

Casos de Prueba — Clases, Funciones, Recursión y Listas

Se implementó una gramática más completa que acepta clases, funciones, llamadas a función, recursión y listas. Este caso demuestra que el sistema puede trabajar con gramáticas más grandes que una simple calculadora aritmética.

Fragmento de la gramática usada en `tests/cases/05_classes_functions/`:

```
program      : decl_list ;
decl_list   : decl_list decl | decl ;
class_decl  : CLASS ID LBRACE func_list RBRACE ;
func_decl   : DEF ID LPAREN param_list RPAREN LBRACE stmt_list
              RBRACE ;
func_call   : ID LPAREN arg_list RPAREN ;
list_expr   : LBRACKET elem_list RBRACKET ;
```

Esta gramática contiene varios elementos interesantes:

- **Recursión:** permite listas de declaraciones mediante reglas como `decl_list : decl_list decl | decl`.
- **Estructuras anidadas:** permite clases con funciones dentro.
- **Llamadas a función:** reconoce expresiones como `foo(1, 2)`.
- **Listas:** reconoce expresiones con corchetes y elementos separados por comas.

Cadenas analizadas y sus resultados:

Cadena	SLR(1)	LALR
<code>x = 1 ;</code>	ACCEPT	ACCEPT
<code>def foo (x) { return x ; }</code>	ACCEPT	ACCEPT
<code>class MyClass { def bar () { return 1 ; } }</code>	ACCEPT	ACCEPT
<code>result = foo (1 , 2) ;</code>	ACCEPT	ACCEPT
<code>items = [1 , 2 , 3] ;</code>	ACCEPT	ACCEPT

Estos resultados muestran que la implementación puede reconocer estructuras comunes en lenguajes de programación, como asignaciones, definición de funciones, clases, llamadas y listas.

Conclusiones

- Se implementó exitosamente un generador de analizadores sintácticos integrado con el generador léxico del proyecto anterior.
- El sistema construye y compara tres métodos de parsing: SLR(1), LALR y LL(1), permitiendo observar diferencias entre enfoques bottom-up y top-down.
- El autómata LR(0) sirve como base para los métodos bottom-up, ya que representa los posibles estados del parser durante el reconocimiento de una entrada.
- FIRST y FOLLOW son esenciales para la construcción de tablas: SLR(1) utiliza FOLLOW para decidir reducciones, mientras que LL(1) utiliza FIRST y FOLLOW para seleccionar producciones.
- La diferencia principal entre SLR(1) y LALR es la precisión del lookahead: SLR(1) usa FOLLOW global, mientras que LALR usa lookaheads más específicos por contexto.
- LL(1) es más simple y fácil de visualizar, pero no maneja recursión izquierda, lo que limita las gramáticas que puede analizar directamente.
- La construcción del árbol de derivación permite comprobar visualmente qué producciones fueron aplicadas para aceptar una cadena.
- El manejo de errores con spans facilita identificar el token exacto que causó el rechazo, mejorando la utilidad del sistema.
- La interfaz gráfica facilita la comprensión del proceso completo al mostrar autómatas, tablas, árboles, pasos y resultados de análisis.